

RAPPORT DE PROJET

Moustaev Djabrail
Ouissi Mohamed
Groupe : 5
07/12/2025
IUT de Fontainebleau

Sujet : SAE 1.01 - Implémentation d'un Taquin en C (89)

I. Introduction

Ce projet a pour but de développer une version numérique du célèbre jeu de Taquin en utilisant le langage C (C89).

Le principe du jeu est simple : l'utilisateur doit reconstituer une image qui a été découpée et mélangée en plusieurs tuiles. Il fait glisser les tuiles dans le cadre une par une, en utilisant l'espace vide, jusqu'à ce que l'image soit complète.

L'objectif principal pour le joueur est de reconstituer l'image correctement en utilisant le moins de mouvements possibles.

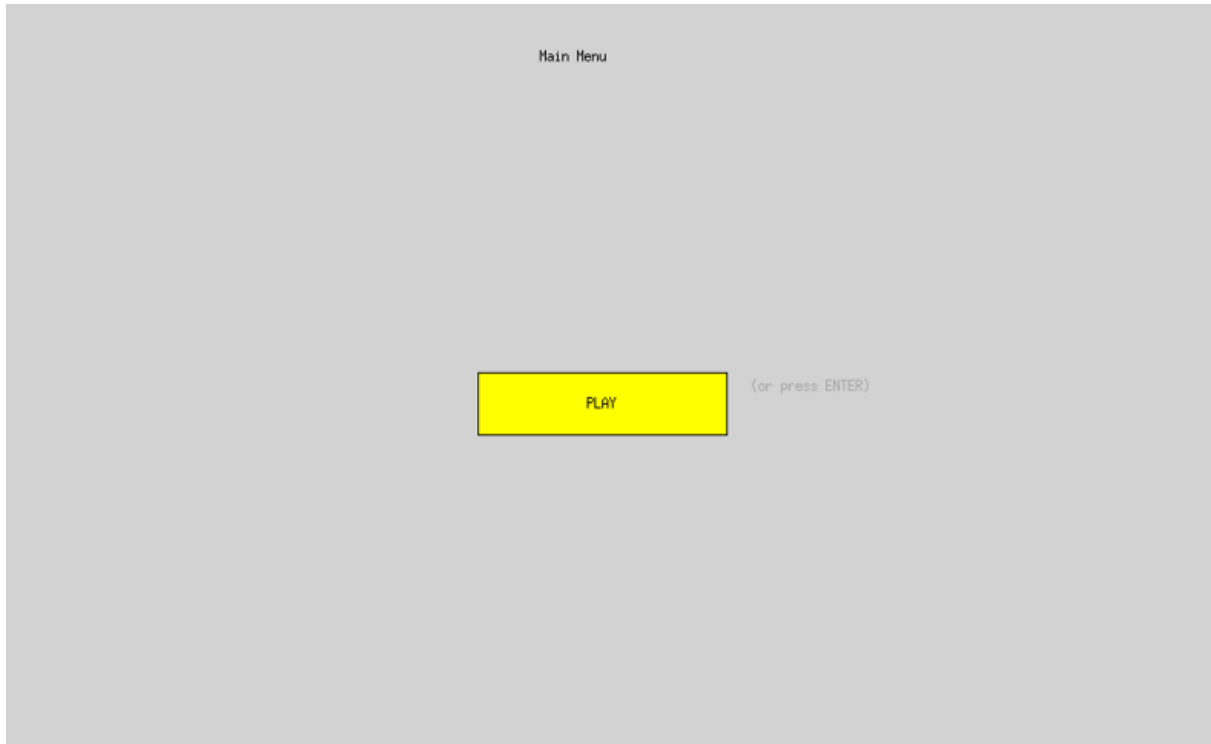
Pour l'affichage et la gestion de l'interface graphique du jeu, ce projet a été réalisé grâce à la bibliothèque graphique de l'IUT. Celle-ci nous a permis d'afficher les tuiles, l'image et de gérer les interactions de l'utilisateur (clics de souris et clavier).

II. Description des Fonctionnalités

Notre programme se divise en plusieurs menus :

A. Menu d'ouverture de jeu

On retrouve ici tout simplement le bouton Play cliquable avec la souris et également on peut appuyer tout simplement Entrer



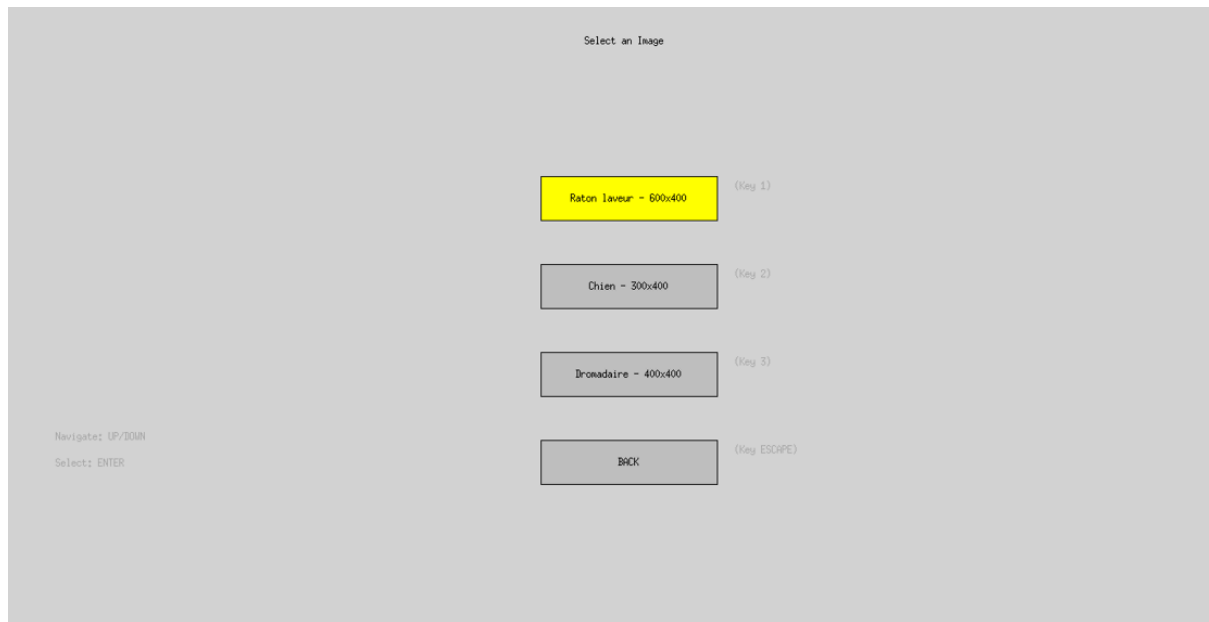
B. Menu de sélection de l'image (Avec les dimensions de ces images)

Ici le joueur doit choisir l'image qu'on vas utiliser pour le jeu de taquin

Il sont affichés avec leurs dimensions

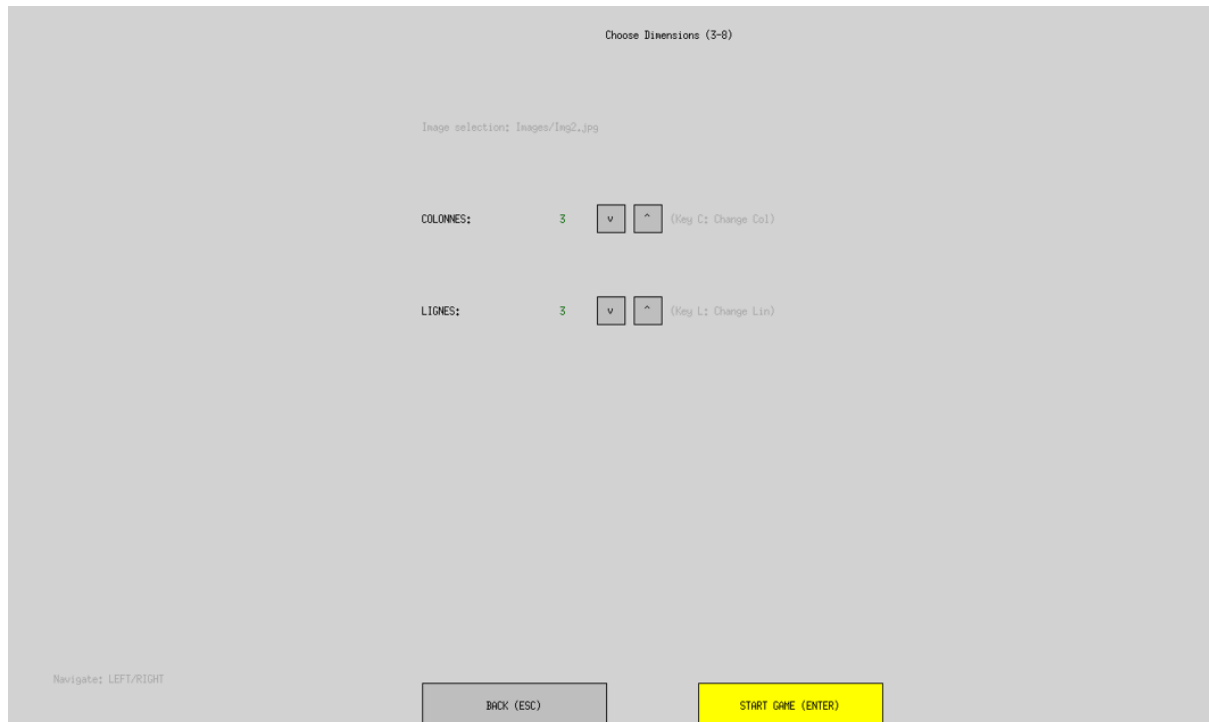
Il peut soit cliquer soit appuyer sur le chiffre correspondant (1,2,3)

Il peut revenir au menu précédent en cliquant le bouton Back (soit Escape)



C. Menu de sélection de nombre de colonnes et de lignes

Ici le joueur choisi le nombre de colonnes et de lignes qu'il aura pendant la partie
Il peut le faire en Augmentant/Diminuant le nombre de colonnes avec la souris en cliquant sur les boutons juste à côté des colonnes/lignes, soit en appuyant C/L pour les colonnes et lignes, elles vont augmenter a chaque appuie du bouton C/L et vont revenir à 3 lorsqu'on dépasse 8.



D. Le jeu de taquin en lui même

La partie se lance:

Déplacement : Le joueur clique sur une tuile adjacente à la case vide pour la déplacer, ou utilise les flèches du clavier.

Interface : Un compteur de coups augmente à chaque mouvement.

- Un bouton "MENU" 'M' permet d'abandonner et revenir à la configuration.
- Un bouton "Shuffle" 'S' permet de mélanger les cases autant de fois qu'on veut
- Un bouton "Reset" 'R' permet de revenir à la position des cases de départ avant avoir fait des déplacements des cases



E. Fin de jeu

La partie est terminée, le joueur peut soit quitter complètement le jeu, soit revenir au menu.

III. Structure du programme

Le programme est séparé en 4 fichiers:

Le code est organisé en quatre fichiers principaux, ce qui permet de séparer la logique de la gestion de l'interface, facilitant ainsi la lecture et la compilation du projet :

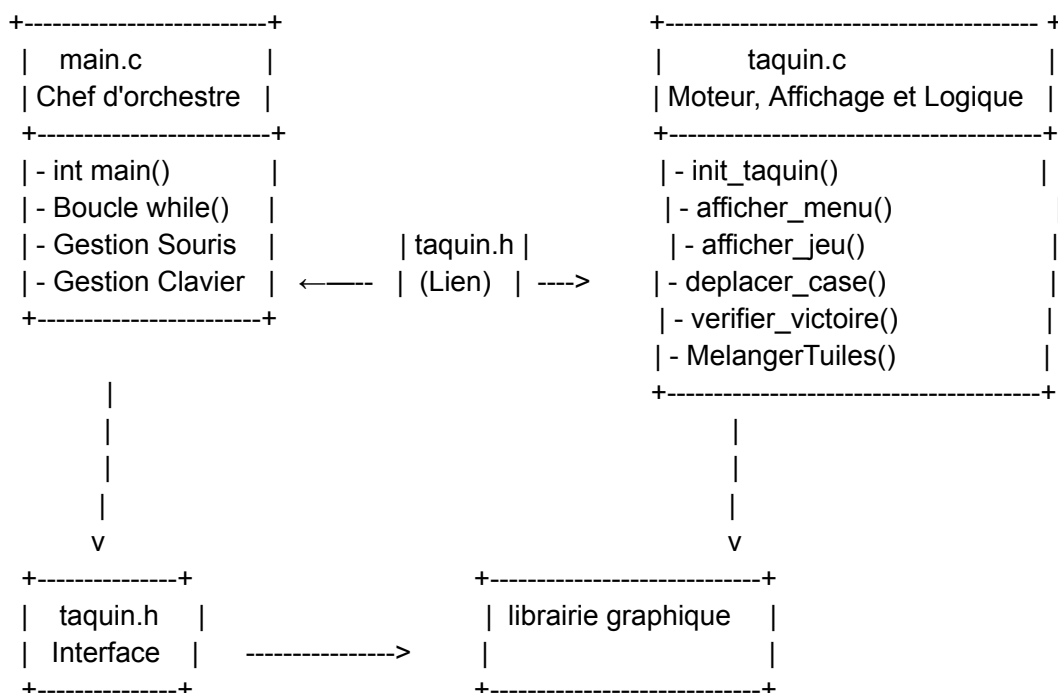
main.c : Il contient le point de départ du programme (int main()) et la boucle principale qui s'exécute en continu. Son rôle est d'orchestrer les transitions d'état entre le Menu et la Partie en utilisant la variable int partieLancee

taquin.c : Ce fichier regroupe toute la logique du jeu. Il contient les fonctions qui gèrent les mouvements de tuiles, le chargement des images, la fonction de mélange (MelangerTuiles), la vérification de la victoire (VerifierVictoire), et toutes les fonctions d'affichage graphique des tuiles et des menus.

taquin.h : Il sert de "pont" entre les fichiers. Il contient les définitions des structures de données (comme les coordonnées de la case vide) et les prototypes de toutes les fonctions du jeu, permettant à main.c d'utiliser les fonctions de taquin.c sans les connaître en détail.

Makefile : Ce script automatise la compilation séparée des fichiers .c et gère l'édition de liens avec la bibliothèque graphique de l'IUT, ce qui simplifie le processus de création de l'exécutable final.

DIAGRAMME



VI. Donnée et Etats du jeu

Pour que le jeu de Taquin fonctionne correctement, notre programme utilise plusieurs variables globales pour stocker l'état du plateau, les choix de configuration, et l'écran actif (menu ou jeu).

Les données et l'état du jeu sont gérés par plusieurs variables.

Pour la **Grille et l'État de la Partie**, nous utilisons la matrice

```
int tuiles[MAX_DIM][MAX_DIM].
```

C'est le cœur du jeu, où chaque case stocke un nombre correspondant à l'index du morceau d'image à afficher (de 1 à N). La valeur **0** représente toujours la **case vide** du puzzle. Une copie, `int initial_tuiles[MAX_DIM][MAX_DIM]`, est conservée pour permettre la fonction **"RESET"**. Pour éviter de chercher le trou à chaque fois, nous stockons ses coordonnées exactes dans `int vide_x` et `int vide_y`, ce qui **optimise** grandement les mouvements. Enfin, `int nombreCoups` est un simple compteur qui suit la progression du joueur.

Concernant la **Configuration de la Partie**, les choix faits dans les menus sont mémorisés. Les dimensions sont stockées dans `int colonnes` et `int lignes`. L'image choisie est enregistrée via son chemin dans `char imageChoisie[80]`. Ces choix permettent ensuite de calculer les dimensions réelles en pixels de chaque morceau, stockées dans `int largeurCase` et `int hauteurCase`, afin d'adapter correctement l'affichage.

La **Machine à États et les Menus** utilise des indicateurs pour gérer les différentes phases du jeu. La variable `int partieLancee` bascule entre **0** (mode Menu) et **1** (mode Jeu actif). Dès que le puzzle est résolu, `int jeuTermine` passe à **1**, ce qui bloque les mouvements et déclenche le **Menu de Fin de Jeu**. Le programme utilise aussi `int menuActif` pour identifier quel sous-menu est affiché (Choix de l'image, Choix des dimensions, etc.), et `int menuSelection` pour indiquer l'option actuellement surlignée par l'utilisateur lorsqu'il navigue entre les options des menus.

Pour l'**Optimisation Graphique et la Sortie**, `int besoinDeReafficherJeu` est crucial : il passe à **1** uniquement après un changement d'état (un mouvement, un bouton pressé) et s'assure que l'écran n'est redessiné que si nécessaire, garantissant la **fluidité**. Enfin, `int jeuDoitQuitter` passe à **1** lorsque l'utilisateur demande explicitement de fermer l'application (via le bouton **QUITTER** du menu de fin), forçant l'arrêt de la boucle principale du programme (`main`).

V. Algorithme de mélange et démonstration de la solubilité

Pour garantir la solvabilité à 100%, notre programme n'utilise pas un placement aléatoire des tuiles. Nous partons toujours de l'état résolu (où toutes les tuiles sont en ordre et le vide est en haut à gauche). La fonction `MelangerTuiles` applique ensuite une longue série (300) de mouvements valides au hasard, en s'assurant que chaque déplacement est techniquement possible (une tuile bouge dans l'espace vide adjacent). Une fois ces

mouvements effectués, la case vide est ramenée à sa position de départ **(0,0)** en utilisant d'autres mouvements valides.

Puisque nous créons l'état mélangé (B) en partant de l'état résolu (A) et en n'appliquant que des mouvements que le joueur pourrait faire, nous savons qu'un chemin de mouvements existe de **A vers B**. Par définition mathématique, si un chemin existe de A à B, le chemin inverse de **B vers A (la solution)** existe aussi. La configuration obtenue est donc obligatoirement solvable.

VI. Conclusion personnelle

Djabrail Moustaeu :

Ce projet de Taquin a été un défi très intéressant pour mettre en pratique nos connaissances en C89. La gestion des tableaux pour créer la grille et surtout le suivi des contraintes du C89 étaient les aspects les plus compliqués. Nous avons beaucoup cherché sur Internet pour comprendre comment fonctionnait la librairie graphique et comment gérer les événements clavier et souris. Voir le code enfin s'animer graphiquement, après avoir passé du temps à se débrouiller face aux erreurs, était vraiment gratifiant. Je suis content du résultat final et ça m'a beaucoup appris sur la nécessité d'être rigoureux avec ce langage.

Mohamed Ouissi:

Pour ma part, l'aspect le plus formateur a été l'autonomie et la résolution de problèmes. Chaque nouvelle fonctionnalité, comme le mélange ou la vérification de la victoire, nous forçait à trouver des solutions originales en C. C'était incroyable de voir toutes les pièces du puzzle s'assembler à la fin, notamment la gestion des états (savoir si on était dans un menu ou en partie) qui était complexe. Malgré les difficultés et les erreurs de parcours, le fait de persister et de voir le jeu fonctionner de A à Z nous rend fiers de notre travail. Ce projet a vraiment permis de mieux comprendre les contraintes et les possibilités du C89 en dehors d'un environnement console